
Spectral Display Basics

Overview

This workshop will use Gnu Radio Companion to demonstrate how to create a basic spectral display, then explain various facets of the spectral display. This includes windowing, normalizing the spectral magnitude values, spectral averaging, and creating a "zoom FFT" display.

If time is still available, then we'll look at overlapped FFTs, the use of the polyphase filter bank (PFB, aka "WOLA" or "weighted-overlap add") spectral display, and creating variable resolution bandwidths.

As we progress through the different exercises, each exercise will build on the previous one.

However, its strongly recommended that you save each exercise as its own file.

Start by downloading the files from Github: `git clone https://github.com/JesterNoFool/Skinnycon2026/`

General order

1. Build a basic spectral display
2. Add a spectrogram
3. Add averaging
4. Provide accurate measurements relative to "full scale"
5. Zoom FFT

If time remains or if there's a lot of interest, we can also look at:

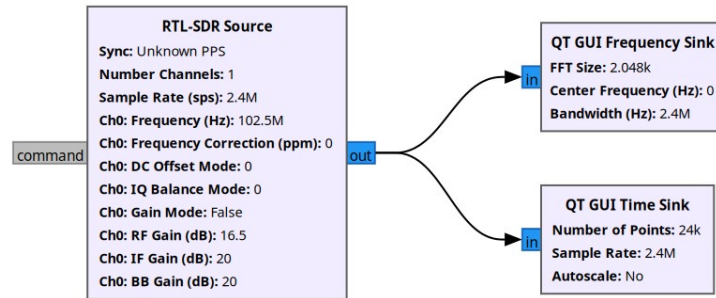
- Overlapping FFTs
- WOLA (weight-overlap-add, aka polyphase filter bank) spectral display
- Variable RBWs

Build a Basic Spectral Display

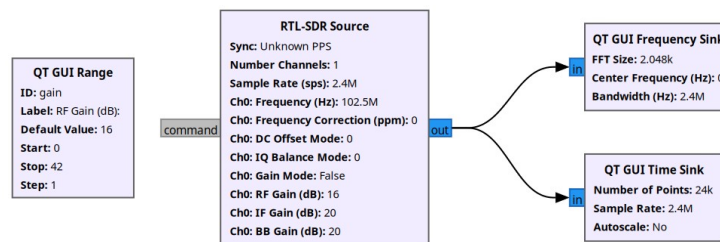
Creating the basic display

1. Start a new flowgraph
2. Save the flowgraph (Save As...). This will also create the Python name in the Options block.
NOTE:
 - A. The name cannot use dashes or spaces, just letters, numbers and underscores.
 - B. The name cannot start with an underscore.
3. Add the RTL-SDR source.
4. Open the RTL-SDR source properties and change its Ch0: Frequency to "102.5e6".
5. Open the properties of the Variable block named "samp_rate" and change the value to 2.4e6.
6. Add a Variable block for the value N (length of time record).
 - A. ID: N
 - B. Value: 2**11. It should show 2048.
7. Add a QT GUI Time Sink and a QT GUI Frequency Sink to the flowgraph.
 - A. Time Sink: Number of Points: 24000
 - B. Freq Sink: FFT Size: N

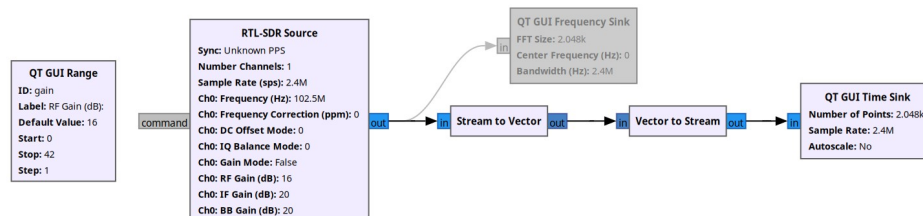
8. Connect the time and frequency sinks to the output of the RTL-SDR.



9. Run the flowgraph to ensure that the RTL-SDR source is working correctly.
10. Add a range block for the gain of the RTL-SDR.
 - A. ID: gain
 - B. Label: Gain (dB):
 - C. Default Value: -1
 - D. Start: -1
 - E. Stop: 42
11. Open the properties for the RTL-SDR source and change the RF Gain value to "gain".



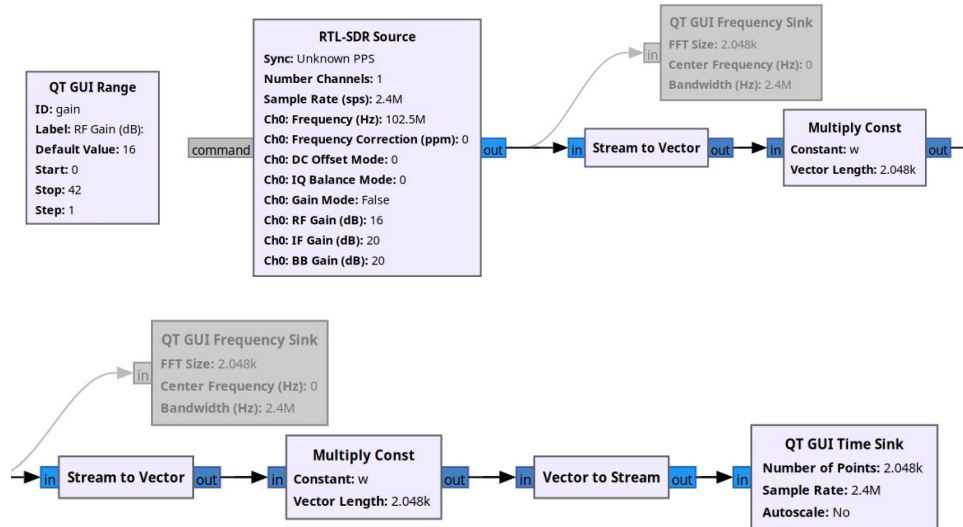
12. Run the flowgraph again and find the best gain for the RTL-SDR with the desired signal.
13. Stop the flowgraph.
14. Set the default in the range block with the measured gain.
15. Disable (Do not delete!) the frequency sink. We'll enable it for comparison shortly.
16. Disconnect the Time Sink from the output of the RTL-SDR Source.
17. Add a "Stream to Vector" block and change the "Num Items" to N. This creates the time record.
18. Connect the Stream to Vector to the output of the RTL-SDR.
19. Add a "Vector to Stream" block and change the "Num Items" to N.
20. Connect the Vector to Stream to the output of the Stream to Vector.
21. Connect the Time Sink to the output of the Vector to Stream, and change the Number of Points in the Time Sink to N.



22. Run the flowgraph (just to ensure it is still working properly).
23. Add the blocks necessary for windowing
 - A. Variable block

- i. ID: w
- ii. Value: window.blackmanharris(N)
- B. Multiply Const block
 - i. Vector Length: N
 - ii. Constant: w

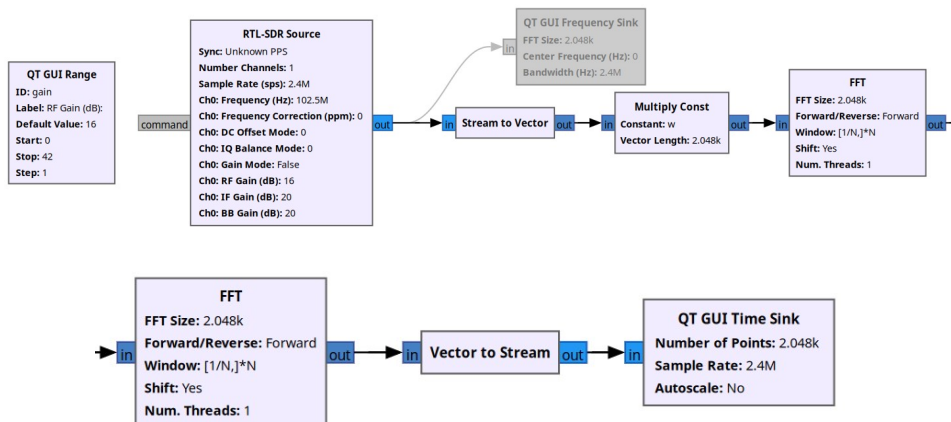
24. Connect the Multiply Const block in between the Stream to Vector and the Vector to Stream.



25. Run the flowgraph. This should show the windowed time record.

26. Add a FFT block between the Multiply Const block and the Vector to Stream block.

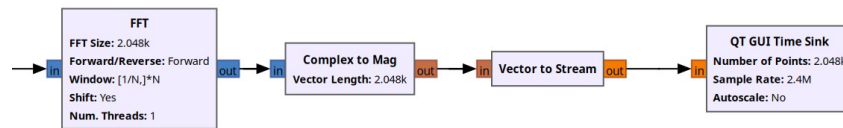
- A. FFT Size: N
- B. Window: $[1/N]*N$. NOTES:
 - i. This normalizes the data levels based on the length of the FFT.
 - ii. Do not forget the comma after the "1/N".



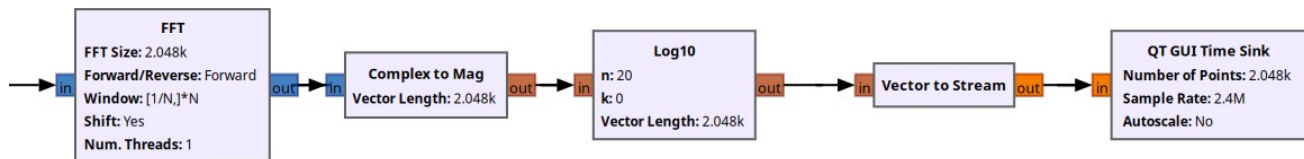
27. Run the flowgraph. This should show the complex output of the FFT. At this point, you can measure the phase and/or the magnitude of each frequency bin.

28. Add a "Complex to Mag" block in between the FFT block and the Vector to Stream block. Set its Vector Length to N.

29. Change the Vector to Stream and Time Sink blocks to accept real-only (aka "float") samples. NOTE: The tabs should be orange on these two blocks.



30. Run the flowgraph. All values should be greater than 0. Also, the values might be very small, requiring a lot of zooming-in to see. These are the "magnitude" measurements, but on a linear scale, for each frequency bin.
31. Add the "Log10" block at the output of the "Complex to Mag" block and before the Vector to Stream block.
 - A. n: 20
 - B. k: 0
 - C. Vector Length: N
32. Change the min and max of the time sink to -140 and 10, respectively.
33. Re-enable the frequency sink.



34. Run the flowgraph. The amplitudes are now on a dB (decibel) scale. The spectrum should be very similar to what is seen on the frequency sink.

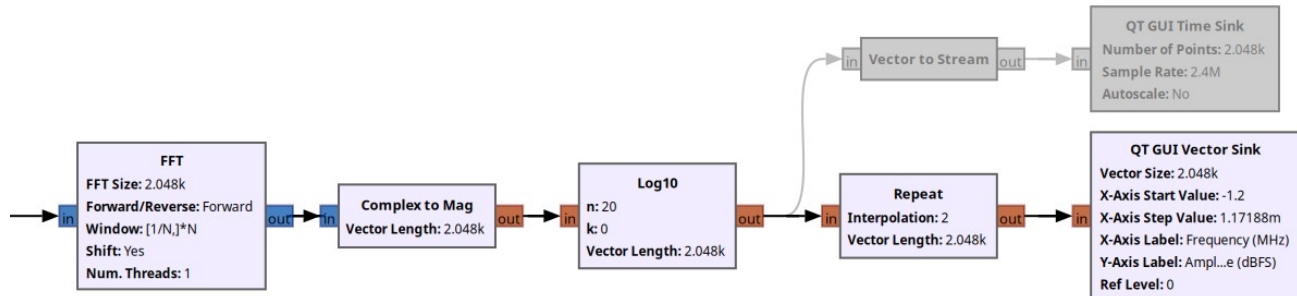
Question for the class: What's missing for making measurements?

- Horizontal scale frequency measurements (start, stop, center)
- Reference level.
- Resolution bandwidth (RBW) value.

Adding a frequency scale

1. Add a "Repeat" and "QT Vector Sink" block to the flowgraph.
2. Disable the Vector to Stream block and the Time Sink block.
3. Connect the "Repeat" block to the output of the "Log10" block. NOTE: This block is a work-around for an issue with Vector Sink in which it will drop samples and display zeros.
 - A. Type: float
 - B. Interpolation: 2
 - C. Vector Length: N
4. Connect the QT Vector Sink to the output of the Repeat block.
 - A. Vector Size: N
 - B. X-axis Start Value: $-\text{samp_rate}/2\text{e}6$. The "samp_rate/2" is for the Nyquist frequency, and the additional 1e6 is to allow for the use of MHz.
 - C. X-axis Step Value: $\text{samp_rate}/N/1\text{e}6$. The "samp_rate/N" is for the binwidth. Again, the 1e6 is for units of MHz.
 - D. X-axis Label: Frequency (MHz)
 - E. Y-axis Label: Amplitude (dB)

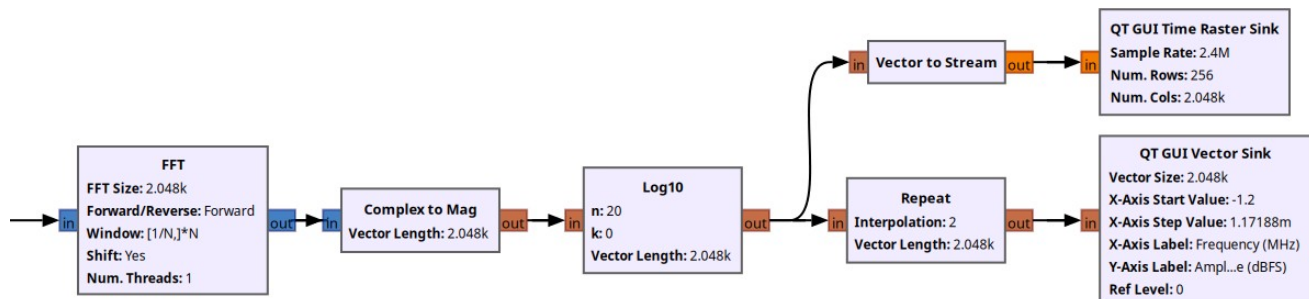
- F. X-axis Units: MHz
- G. Y-axis Units: dB
- H. Grid: Yes
- I. Y-min: -140
- J. Y-max: 10
- K. Update Period: 0.0001



5. Run the flowgraph. The result should, again, be very close to what is seen in the Frequency Sink. The horizontal scale should be equivalent to what is seen in the QT GUI Frequency Sink.
6. Stop the flowgraph.
7. OPTIONAL: Change the X-axis Start Value in the QT GUI Vector Sink to: $-\text{samp_rate}/2\text{e}6 + \text{cf}$, where "cf" is the center frequency of the RTL-SDR, in MHz.
8. Run the flowgraph. The horizontal scale should show the frequencies of the various signals correctly.

Add a Spectrogram

1. Re-enable the Vector to Stream block connected to the output of the Log10 block.
2. Add a "Time Raster Sink" block. Connect the Time Raster Sink to the output of the Vector to Stream block.
 - A. Type: Float
 - B. Num Cols: N
 - C. Int. Min: -140
 - D. Int. Max: 10
 - E. Update Period: 0.0001
3. Run the flowgraph. The Time Raster Sink should display a rising waterfall.



Normalize the amplitude measurements

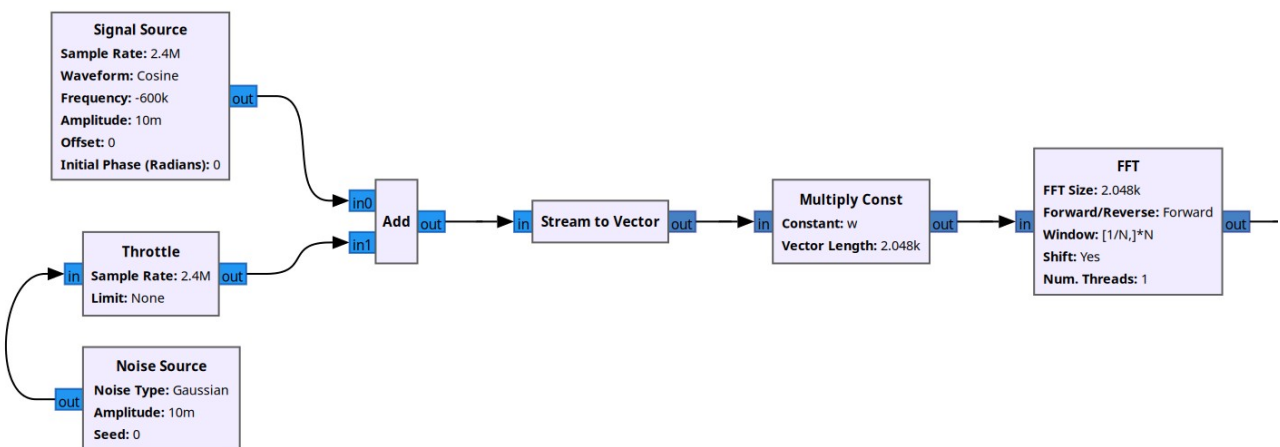
Add Averaging

In order to make better amplitude measurements, we're going to first add averaging to our display. This requires some understanding of where to put the averaging, since doing it in the wrong spot will give you incorrect measurements. Where to place the averaging boils down to two questions:

1. Do you average the log value, or do you take the log value of the average?
2. Do you average the square, or do you square the average?

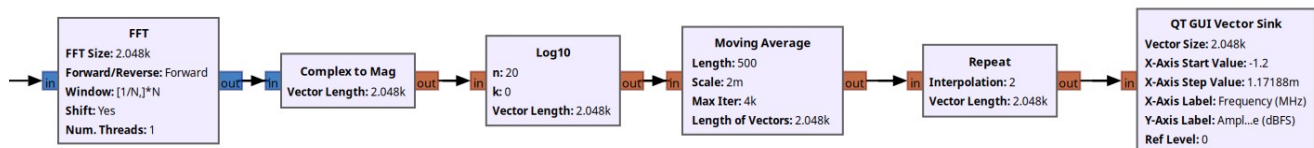
To try these different methods, we'll perform the following steps:

1. Save the flowgraph under a new name, with "average" in the name... somewhere.
2. Disconnect the RTL-SDR Source block from the Stream to Vector block, and then disable the RTL-SDR Source.
3. Add an Add block to the flowgraph.
4. Connect the output of the Add block to the input of the Stream to Vector block.
5. Add a Signal Source block to the flowgraph and connect it to the first input of the Add block.
 - A. Frequency: $-\text{samp_rate}/N*(N/4)$
 - B. Amplitude: 0.1
6. Move the input of the QT GUI Frequency Sink to the output of the Add block.
7. Delete the frequency sink.
8. Add a Throttle block and Noise Source block.
 - A. Noise Source output connected to Throttle input
 - i. Amplitude: 0.01
 - B. Throttle block output connected to the second Add input.

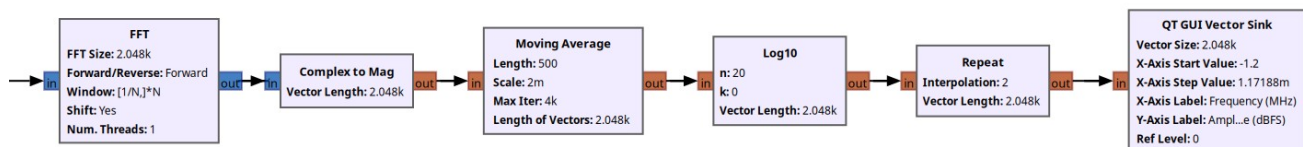


9. Add an Import block with the value of "import numpy as np".
10. Add three Variable blocks.
 - A. Variable #1
 - i. ID: num_ave

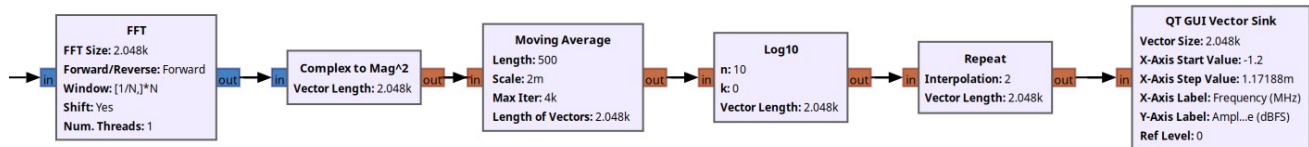
- ii. Value: 500
- B. Variable #2
 - i. ID: An
 - ii. Value: $20 * \text{np.log10}(0.01) - 10 * \text{np.log10}(N)$
- C. Variable #3
 - i. ID: As
 - ii. Value: $20 * \text{np.log10}(0.1)$
- 11. Add a Moving Average block to the flowgraph.
 - A. Type: float
 - B. Length: num_ave
 - C. Scale: $1/\text{num_ave}$
 - D. Length of Vectors: N
- 12. Connect the Moving Average block in between the Log10 and Repeat blocks.
- 13. Open the properties for the QT GUI Vector Sink and make the following changes:
 - A. Y min: -90
 - B. Y max: -10
- 14. Change the "w" variable to "window.rectangular(N)".



- 15. Run the flowgraph.
- 16. With the flowgraph running, measure:
 - A. the amplitude of the noise.
 - B. the amplitude of the peak of the spectral line.
- 17. Stop the flowgraph.
- 18. Move the Moving Average block to between the Complex to Mag and Log10 blocks.



- 19. Run the flowgraph and measure the same points as before.
- 20. Stop the flowgraph.
- 21. Replace the Complex to Mag with a Complex to Mag² block. Change the "Vector Length" in the Complex to Mag² block to "N".
- 22. Change the "n" value in the Log10 block to "10".



23. Run the flowgraph and measure the same points as before.

The students should have different measurements for each. The question is, "Which is the one that provides the closest values to both the sinusoid and noise?" The answer should be that the Moving Average fits best between the Complex to Mag² and the Log10 blocks. This means you want to:

- take the average of the square
- take the log of the average

Now that we have the averaging in the correct place, let's look at the windowing and its effects.

How Windows Affect the Amplitudes

This will show how the amplitudes change based on the window used.

1. In the Variable block for "w", change the value to "window.hann(N)".
2. Run the flowgraph and measure the amplitude at the same places (noise and spectral line) as before.
3. Stop the flowgraph.
4. Repeat steps 1 - 3 with the "blackmanharris" and "flattop" windows for the "w" value.

Compare the various values of noise and spectral line amplitudes for the different window types.

- What are the differences between the noise amplitudes?
- What are the differences between the spectral line amplitudes?
- Are these differences the same between the noise and spectral line?

Normalize the Amplitudes

The differences between power levels of the spectral line and the noise floor is due to using different windows. Each window subtracts a set amount of power from the signal before being put into the FFT. (NOTE: Windowing reduces the SNR of the signal on the spectral display, but it more than makes up for it by reducing spectral leakage and scalloping loss.) The windows affect the power differently depending on whether the signal is coherent (spectral line) or incoherent (noise or noise-like). We can adjust for these measurements using the following:

- Coherent power gain: This normalizes the peak amplitude and is designed for sinusoids and other coherent signals.
- Incoherent power gain: This normalizes the bin amplitudes so that the total power for a signal (the power summed over each frequency bin) is correct.

Both of these values can be calculated from the window values.

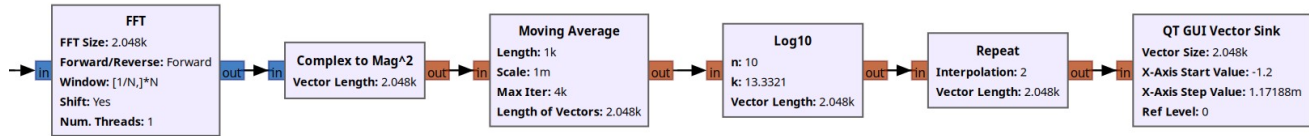
1. Add three Variable blocks to the flowgraph:
 - A. Variable (coherent power gain)
 - i. ID: cpg
 - ii. Value: $(np.sum(w)/N)**2$

B. Variable (incoherent power gain)

i. ID: ipg

ii. Value: $\text{np.sum}(\text{np.power}(w,2))/N$

2. Open the properties for the Log10 block and change the "k" value to $-10*\text{np.log10}(\text{cpg})$.



3. Run the flowgraph and measure the peak of the sinusoid and the level of the noise.

4. Stop the flowgraph.

5. Changes the windows to "blackmanharris", "hann", and "rectangular", and re-run the flowgraph after each change.

6. For each run, measure the sinusoid and noise level.

The result of these measurements should show that the amplitude of the spectral line is the same for all windows, but the noise (the incoherent power) measurements are different for each. If the "k" value in the Log10 block were for the incoherent power gain ($-10*\text{np.log10}(\text{ipg})$), then the noise levels would have been (roughly) similar, while the spectral line amplitudes would have been different.

We can use either and calculate the other using the window factor. In this case, the window factor is the normalized equivalent noise bandwidth (NENBW). It can be calculated as $\text{NENBW} = \text{IPG}/\text{CPG}$.

Table 1: This table shows values for some common windows used in Gnu Radio Companion. Note that these values might differ slightly from those calculated in GRC due to the fact that these were calculated in Gnu Octave, which by default uses double precision numbers. GRC defaults to single precision. Therefore, these are more accurate than those provided in GRC.

Window	IPG	CPG	NENBW	Scalloping Loss (dB)
Rectangular	0	0	1.0000	3.92
Hann(ing)	0.3750	0.2500	1.5000	1.42
Blackman-harris	0.2580	0.1287	2.0044	0.83
Flattop	0.1754	0.046528	3.7702	0.016

This leads to some basics of spectral displays when normalized with either the incoherent power gain (IPG) or coherent power gain (CPG).

- **If corrected for the IPG, then the power distribution of noise will be power/bin.** In other words, you can calculate the total noise by measuring the DANL (displayed average noise level) of a display (assuming that it is flat) and multiplying the measured power by the number of points in the FFT. If the signal bandwidth is greater than the RBW, then the signal behaves as incoherent.
- **If corrected for the CPG, then the power distribution will be power/Hz.** We can calculate the total noise by measuring the DANL of a display (again, assuming it is flat), dividing by the

ENBW and multiplying by the sample rate. Another option is to convert the DANL from power/Hz to power/bin by dividing by the NENBW. The total noise can then be calculated by multiplying by the number of points in the FFT. If the signal bandwidth is less than the RBW, then the signal behaves as coherent.

Most high-end systems use CPG for normalizing the values, as do several of the most-popular open source SDR programs, such as GQRX, SDRangel, and SDRconnect. Perhaps the reason why is that it makes them closer to how older, analog systems operated. For example, if you wanted to measure the power of a noise-like signal, you could increase the RBW (which is the same as the ENBW normally) until it was equal to, or slightly greater than, the signal bandwidth. If the amplitudes are normalized with CPG, then the total power will be the measured peak of the signal.

Further, using CPG, if you want to measure the band power by adding up the power of the frequency bins that make up that signal, you can add them up then divide by the NENBW. This final division turns the CPG into IPG and the band power measurement will be accurate.

Lastly, CPG together with the RBW (typically calculated as the NENBW), which is more likely to be information provided to the user, can provide the power spectral density in power/Hz. This is versus the number of points in the display, something that is not (normally) provided.

However, not everyone uses this methodology. For example, the Gnu Radio Companion QT GUI Frequency Sink has a property setting for "Normalize Window Power". If selected, this will normalize the power of each frequency bin based on the IPG.

This leads us to one, final mention about the amplitudes of our display: They're normalized but not calibrated. If you ever wonder why certain systems are so expensive, or why "These SDRs are so cheap. Why don't I just use them instead of those expensive systems?", one of the big reasons is calibration. In order to have a system that is calibrated within a defined tolerance, the system has to be designed for it AND the system has to be tested against a known calibration system. Both of these cost money. For example, think about the gain of the LNA (low noise amplifier). Currently, as we adjust the gain of that LNA, the entire spectrum will shift up or down depending on whether we increase or decrease the gain of the LNA. For a calibrated system, we would need to account for the gain adjustment since the measured power of an input signal should not be affected by the gain of the amplifier. For example, we could add a block that adjusts the amplitude of the spectral display to account for the RTL-SDR RF gain. This would, however, no longer allow the scale to be "dBfs" (decibels relative to full scale), and would require actually calibrating the system.

Scalloping Loss

A major issue with selecting a window to use is the concept of "scallop loss". This is the change in the amplitude of a coherent signal, such as a sinusoid, based on its location within a frequency bin. This cannot be normalized, only minimized based on the window used. The largest error occurs with no window (rectangular, boxcar, uniform), while the smallest occurs with a family of windows called "flattop".

1. In the properties for the Signal Source, change the "Frequency" to " $-\text{samp_rate}/N \cdot (N/4 + 0.5)$ ".
2. Run the flowgraph.
3. Measure the amplitude of the spectral line.

4. Stop the flowgraph.
5. Rerun the flowgraph with the "w" variable changed to "window.hann(N)", "window.blackmanharris(N)", and "window.flattop(N)". Measure the spectral line amplitude for each.

Note that scalloping loss is only an issue with the amplitudes of coherent signals; incoherent signals are not affected by this.

Adding the Resolution Bandwidth (RBW)

Up til now, we've added two of the three main spectral display parameters (reference level, frequency limits). We're still missing the third. That's the resolution bandwidth. The form of RBW that we will calculate is the equivalent noise bandwidth (ENBW). This is the equivalent bandwidth that captures the same amount of noise as if we had a perfect, brickwall filter. Fortunately, we have everything we need to calculate the RBW from the various values we've already calculated or already have. These are:

- IPG
- CPG
- Sample rate (fs)
- Number of points in the FFT (N)

The calculation is as follows:

$$\text{RBW (ENBW)} = \frac{f_s \cdot \text{IPG}}{\text{CPG} \cdot N}$$

The value IPG / CPG is normally calculated as the normalized equivalent noise bandwidth (NENBW). Thus, the calculation for the RBW becomes:

$$\text{RBW (ENBW)} = \frac{f_s \cdot \text{NENBW}}{N}$$

The value f_s/N is the binwidth. So the RBW can also be expressed as the binwidth multiplied by the NENBW.

Note that RBW is distinct from binwidth (which, again, is just the sample rate divided by the number of points in the FFT). The resolution bandwidth is dependent on the window used; some programs just declare the binwidth as the RBW (Looking at you, GQRX!)

Anyway, to add the RBW to our flowgraph:

1. Add a Label block to the flowgraph
 - A. ID: nenbw
 - B. Label: NENBW:
 - C. Type: Float
 - D. Default Value: ipg/cpg
2. Add a Label block to the flowgraph
 - A. ID: rbw
 - B. Label: Resolution Bandwidth (RBW, Hz):
 - C. Type: Float
 - D. Default value: samp_rate*nenbw/N

Now, any time you change any of the requisite values, the RBW will be recalculated.

Choosing a Window

Choosing the window to use is based on three aspects. These are:

- Scalloping loss
- Resolution bandwidth
- Sidelobe attenuation

You may need to select different windows for the same signal depending on what you're trying to measure. Therefore, having the ability to adjust windows should be a prerequisite for a well-functioning spectral display.

There are also times when you will not want a window at all. The main time this occurs is when dealing with burst / pulsed signals. Such signals are said to be "self-windowed", due to the fact that they go to zero naturally (when they're not transmitting).

Measuring the SDR ENOB (Effective Number of Bits)

The effective number of bits (ENOB) is a unit of measure for the digitizer of a digital system. It's closely related to another measure called "SINAD", which stands for "signal-to-noise-and-distortion". SINAD is not just a measure of noise; it also includes things such as the spurious signals created in both the analog and digital portions of a system. When measuring the DANL of a system, that is also a measure of its SINAD. This means it's possible to get a (rough) measure of a system's ENOB by measuring its DANL, then using this to calculate its total noise. The total noise will be the SINAD, which can then be used to calculate its ENOB.

1. Save the flowgraph under a new name, such as "improved_spectral_display".
2. Delete the Signal Source, Noise Source, Throttle and Add blocks.
3. Re-enable the RTL-SDR Source block.
4. Disconnect the antenna from the RTL-SDR.
5. Open the properties for the Log10 block and change the "k" value to "-10*np.log10(ipg)".
6. Change the "num_ave" value to 1000.
7. Change the window type to whatever you want.
8. Run the flowgraph.

Before continuing on, let's take a moment to view this noise floor and what it tells us. There are four aspects of interest in this display of the noise floor. These are:

- The relative noise floor itself: This is primarily due to the quantization noise of the digitizer. It creates a relatively flat, even noise floor.
- The dropoffs at each end of the spectrum: This is due to the anti-aliasing filter that comes before the digitizer. Systems that provide sweeping spectral displays will take great pains to ensure that this is as flat as possible to avoid "drops" or "drop outs" in the spectrum.
- The peak at the center: This is due to how this particular RTL-SDR operates. It uses a RF front end chip (the E4000) that translates the RF signal down to baseband and provides a complex,

analog output. This creates a condition called "LO feedthrough" or "LO leakage". That's the spike at the center.

- The bandwidth of the center peak: The center peak not only shows LO feedthrough, but also phase noise and/or 1/f noise due to the oscillators used in the RF front end.

Continuing on:

9. Measure the noise floor at a point where it is roughly flat. For the E4000-based RTL-SDRs, this is typically roughly half-way between the center of the spectral display and the edge. Because we've normalized the spectral magnitudes based on the IPG, this measured value is the power spectral density (PSD) in power/bin.
 - A. To calculate the (approximate) total noise, add $10 \cdot \log_{10}(\text{FFT size})$. For example, if the measured DANL is -76 dB using the IPG to normalize the spectral magnitudes and the FFT size is 2048 points, then the total noise will be $-76 + 10 \cdot \log_{10}(2048) = -76 + 33.1 = -42.9$ dBFS.
 - B. The absolute value of this (42.9 dBFS) tells us the best SNR we would expect without any further processing. The dynamic range of a digitizer is $6.02n + 1.76$, where "n" is the number of bits/sample. We can use the measured noise power to calculate the effective number of bits (ENOB) as $6.02n + 1.76 = 42.9$. Solving for n, we get $n = 6.8$ bits. Thus, the RTL-SDR (based on the RTL2832U chip, which uses a 8-bit digitizer) has an ENOB of 6.8 bits.
10. Stop the flowgraph.
11. Open the properties for the Log10 block and change the "k" value to $-10 \cdot n \cdot \log_{10}(\text{cpg})$.
12. Run the flowgraph.
13. Again, measure the noise floor at a point where it is roughly flat.
 - A. To calculate the PSD in power/Hz, subtract $10 \cdot \log_{10}(\text{ENBW}^1)$. NOTE: This is the PSD that is almost always used in RF literature and products. In other words, when looking at system specifications for a spectrum analyzer and the specification says the "DANL" is in "PSD", this is the PSD to which they are referring. Ex: The REI OSCOR Max specifies a "DANL: -150 dBm/Hz". This is the PSD referred to here.
 - B. To calculate the total noise, add $10 \cdot \log_{10}(\text{sample rate})$ to the PSD. This value should be almost identical to the total noise value calculated using the IPG above. The difference is that, again, most systems use CPG for normalizing magnitudes, and provide both the RBW (calculated as the ENBW) and the PSD in power/Hz.

Adding a "Zoom" Spectral Display

There are three factors in the RBW. These are the sample rate, the number of points in the FFT, and the window factor (normally calculated as the NENBW). The sample rate and the number of points are typically much larger factors in the RBW, since the NENBW is a value that rests between 1 - 5, with values of 1.5 - 4 being typical. The sample rate can be millions or billions, and the number of points hundreds to millions. Therefore, it's these latter, two values that are typically used to create a spectral display that is zoomed in on a portion of the spectrum.

1 Remember that the ENBW is typically listed as the RBW, so long as the RBW includes the window factor.

1. Reconnect the antenna.
2. Open the properties for the Signal Source and set the amplitude to "0".
3. Change the window ("w" variable) to "window.blackmanharris(N)".
4. Change the number of averages (num_ave) to 10.
5. Run the flowgraph.
6. Measure the center frequency of one of the visible IBOC signals.
7. Stop the flowgraph.
8. Add the following blocks to the flowgraph:
 - A. QT GUI Entry
 - i. ID: fc
 - ii. Label: Zoom Center Frequency (Hz):
 - iii. Type: float
 - iv. Default Value: <the value you measured>
 - B. Rotator
 - i. Phase Increment: $-2 * \pi * f_c / \text{samp_rate}$
 - C. FFT Low Pass Filter
 - i. Decimation: 16
 - ii. Cutoff Freq: 50e3
 - iii. Transition Width: 25e3
9. Connect the three blocks in the following order:
 - A. Rotator input to the RTL-SDR Source block output.
 - B. FFT Low Pass Filter input to the Rotator output.
 - C. Enable and move the QT GUI Frequency Sink input to the FFT Low Pass Filter output.
 - D. In the QT GUI Frequency Sink:
 - i. Change the FFT Size to 32768.
 - ii. Change the Bandwidth (Hz) to "samp_rate/16".
 - iii. Change the Average to High
10. Run the flowgraph. You should see small spectral spikes or dips indicating the spacings between the IBOC channels.
11. You can also add another low pass filter (with decimation) after the first one in order to reduce the sample rate even more.

Overlapping FFTs

This will create several overlapping FFTs to create a display. However, due to the overlapped FFTs, burst / pulsed signals will be more likely to be captured. Note that this is not the same as a truly-overlapped FFT setup. In such a setup, the windows between sections would truly overlap. This setup has a non-overlap between blocks. This means that pulsed / burst signals will dip if they occur in the non-overlap area.

1. Save the flowgraph under a new name, such as "overlapped_spectral_display".
2. Highlight the blocks from the Stream to Vector to the last block.

3. Move the blocks to the right so that there's enough room to fit an additional block between the Add block and the Stream to Vector block.
4. Highlight the blocks from the Stream to Vector block to the Complex to Mag² block.
5. Press ctrl+c to copy these blocks into memory (aka "the clipboard").
6. Press ctrl+v to add the copies to the flowgraph.
7. Move the added blocks just below the original ones.
8. Repeat steps 5 - 6 two more times so that there are a total of 4 streams.
9. Add 3 Delay blocks.
10. Connect the 3 Delay blocks to the output of the Add block.
 - A. Delay block 1: Delay: $\text{int}(N/4)$
 - B. Delay block 2: Delay: $\text{int}(N/2)$
 - C. Delay block 3: Delay: $\text{int}(N*3/4)$
11. Connect 1 Delay block each to the Stream to Vector blocks for each of the 3 new streams.
12. Add a Stream Mux block.
 - A. Type: float
 - B. Lengths: (1,1,1,1)
 - C. Num Inputs: 4
 - D. Vector Length: N
13. Connect the output of each of the Complex to Mag² blocks to the input of the Stream Mux block. (NOTE: This will require disconnecting the first Complex to Mag² block from the Moving Average block.)
14. Connect the output of the Stream Mux block to the input of the Moving Average block.
15. Change the "num_ave" to 20.
16. Run the flowgraph. The spectrum should appear normal.
17. Stop the flowgraph.
18. Add a Vector Source block and a Throttle block.
19. In the Vector Source, change the Vector to $[1,]*20+[0,]*2000$
20. Connect the Vector Source to the Throttle input.
21. Connect the Throttle output to the same input of the Add block as the RTL-SDR.
22. Disable the RTL-SDR.
23. Run the flowgraph. Note how much the signal changes in amplitude.
24. Stop the flowgraph.
25. Disable the three streams passing through the Delay blocks (from the Delay block to the Complex to Mag²).
26. Disable the Stream Mux block.
27. Connect the first Complex to Mag² block to the Moving Average block.
28. Run the flowgraph. Note how much the signal is changing in amplitude.
29. Stop the flowgraph.
30. Change "w" (the window) to "window.rectangular(N)".
31. Run the flowgraph. Again, note how much the signal is changing in amplitude.

This shows how burst / pulsed signals can be handled in different ways. The most straightforward way to handle such signals is by changing the window to rectangular (aka uniform or boxcar). However, there's still the possibility that the burst / pulsed signal will split between two time records. An overlapped set-up alleviates this, and ensures that all such pulsed signals will be captured. However, an overlapped setup requires a lot more processing power. The greater the overlap, the more processing power required.

WOLA (weighted-overlap add) Spectral Display

The basic spectral display uses a window to reduce scalloping loss, spectral leakage and improve dynamic range. This can be combined with overlapped spectral displays to provide certain benefits. By replacing the standard overlapped-and-window scheme with a polyphase filter bank, we achieve certain benefits. These include improved dynamic range, less scalloping loss, and improved performance against burst signals. The downside is that it requires more processing and decreases the ability to see short-time details.

From the downloaded Github files, open `wola_spectral_display.grc` file in Gnu Radio Companion.

Variable RBW Spectral Displays

As discussed previously, the resolution bandwidth (RBW) is based on three factors. These are:

- the sample rate
- the number of samples passed to the FFT
- the window factor

Of these three factors, the window factor has the smallest variable range. Further, the sample rate is typically defined by the need to cover a particular amount of spectrum, or the need to use the highest sample rate in order to achieve the greatest possible processing gain by spreading the quantization noise over the greatest range. This leaves the number of samples passed to the FFT.

For example, assuming a sample rate of 5 MHz and a Blackman-harris window ($NENBW = 2.0044$), and a desired RBW of 5 kHz. The calculation is:

$$N = \frac{f_s}{RBW} \cdot \beta$$

where:

- N : the required number of samples
- f_s : the sample rate
- RBW : the desired RBW
- β : window factor (typically $NENBW$)

The calculation thus becomes:

$$N = \frac{5e6}{5e3} \cdot 2.0044 = 2004.4 = 2004$$

Since the number of samples fed into the FFT must be an integer number, the calculated value can be either rounded down or rounded up (or just rounded to the closest integer).

This is not a power-of-2. While being a power-of-2 is not an inherent requirement for the FFT (different FFT algorithms, such as the Winograd or the Good-Thomas, can operate on different FFT sizes), most of the most efficient sizes are powers-of-2. Therefore, an option to make this a power-of-2 is to add zeros (zeropad) the time record to make it a power-of-2. The next power-of-2 that is greater than 2004 is 2048. Thus, adding 44 zeros will make the time record 2048 points ($2004 + 44 = 2048$). Zeropadding does two things. The first is that it makes it possible to have a variable RBW while still using a power-of-2 FFT algorithm. The second is that it separates the binwidth and the RBW. Without

zeropadding, the RBW is the binwidth multiplied by the window factor. But with zeropadding, the binwidth is the total number of zeropadded samples (again, typically a power-of-2) fed into the FFT. The RBW is based on only the number of actual data samples (before zeropadding) going into the FFT. In this case, the number of data samples is 2004, while the total number of samples is 2048. Thus, the RBW is 5 kHz, while the binwidth is $(5\text{e}6)/2048 = 2441$ Hz. We'll look at a basic flowgraph that provides for variable RBWs. From the downloaded Github files, open `variable_RBW_spectral_display.grc`.

References

- "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform", fredric j. harris, Proceedings of the IEEE, Vol. 66, No. 1, January 1978.
- "Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows.", G. Heinzel, et al, Max-Planck-Institut für Gravitationsphysik, Teilinstitut Hannover, February 15, 2002.
- "Window Functions and Their Applications in Signal Processing", K.M.M. Prabhu, CRC Press, 2014
- "Understanding FFTs and Windowing", National Instruments, URL: <https://download.ni.com/evaluation/pxi/Understanding%20FFTs%20and%20Windowing.pdf>
- "Some Windows with Very Good Sidelobe Behavior", Albert H. Nuttall, IEEE Transactions on Acoustics, Speech, and Signal Processing, Vol. ASSP-29, No. 1, February 1981
- "Understanding Digital Signal Processing (Second Edition)", Richard G. Lyons, Prentice Hall, 2004.
- "Digital Signal Processing: A Practical Guide for Engineers and Scientists", Steven W. Smith, Newnes, Demystifying Technology Series, 2003, URL: <https://www.dspguide.com/pdfbook.htm>
- "Simple but Effective Spectrum Averaging", Neil Robertson, 2 Jan 2026, URL: <https://www.dsprelated.com/showarticle/1773.php>